

PROJECT REPORT

Dynamic Food Delivery Application using MERN Stack

Project Code: FS07P5A

Domain: Full Stack Development

Submitted to: Rixi Lab Technologies

Submission Date: September 8, 2026

1. Introduction

FoodExpress is a dynamic, responsive food delivery web application built using the MERN stack (MongoDB, Express.js, React.js, and Node.js). The platform allows customers to browse restaurants, explore menus, place orders, make payments, and track deliveries in real time.

This project was completed as part of the Full Stack Development program (Project Code: FS07P5A) issued by Rixi Lab Technologies. It demonstrates practical, end-to-end application of RESTful API design, database modeling, authentication, real-time communication, and payment gateway integration.

1.1 Project Objective

The objective of this project is to design and implement a full stack food delivery application that allows users to explore restaurants, browse menus, place orders, and track deliveries in real time, while covering key full stack development concepts such as RESTful APIs, database management, real-time updates, and frontend-backend integration.

1.2 Scope

- User registration, login, and authentication using JWT
- Restaurant and menu browsing with a responsive UI
- Shopping cart and order placement workflow
- Real-time order status tracking using WebSockets
- Secure online payments via Razorpay
- Cloud deployment of both frontend and backend

2. Technology Stack

Layer	Technology	Purpose
Frontend	React.js, React Router, Axios	UI, client-side routing, API calls
Backend	Node.js, Express.js	REST API server, business logic
Database	MongoDB, Mongoose	Data persistence and modeling
Authentication	JSON Web Tokens (JWT), bcrypt.js	Secure login and password hashing
Real-time	Socket.io	Live order status updates
Payments	Razorpay	Order payment processing
Testing	Jest, Supertest	Automated API and middleware testing
Deployment	Vercel (frontend), Render (backend), MongoDB Atlas	Cloud hosting
Version Control	Git & GitHub	Source control, sprint-based history

3. System Architecture

The application follows a client-server architecture with a clear separation of concerns:

- Client (React): renders the UI, manages local state (cart, auth) via React Context, and communicates with the backend over REST and WebSocket connections.
- Server (Express): exposes REST endpoints under /api, handles authentication, business logic, and payment verification, and pushes real-time events via Socket.io.

- Database (MongoDB): stores users, restaurants, menu items, and orders as documents, related via ObjectId references (e.g., an order references its user and menu items).

Data flow example — placing an order:

- 1. User adds items to cart (client-side state via CartContext).
- 2. User places the order → POST /api/orders creates an Order document.
- 3. Client requests a Razorpay order → POST /api/payments/create-order.
- 4. Razorpay Checkout collects payment; on success the client calls POST /api/payments/verify.
- 5. Server verifies the payment signature, updates order status, and emits an orderStatusUpdate event via Socket.io.
- 6. The order tracking page updates instantly without a page refresh.

4. Database Design

4.1 User

name, email (unique), password (hashed), phone, address, role (customer/admin), timestamps.

4.2 Restaurant

name, cuisine, image, rating, address, isOpen, timestamps.

4.3 MenuItem

restaurant (ref → Restaurant), name, description, price, image, category, isAvailable, timestamps.

4.4 Order

user (ref → User), items [{ menuItem (ref → MenuItem), quantity, price }], totalAmount, status (placed / preparing / out_for_delivery / delivered / cancelled), paymentStatus (pending / paid / failed), timestamps.

5. Module Description

5.1 Authentication Module

Handles user signup and login. Passwords are hashed with bcrypt before storage; on successful login a signed JWT is issued and stored client-side, then attached to subsequent API requests via an Axios interceptor. Protected routes use middleware that verifies the token and, where needed, checks the user's role (e.g., admin-only order status updates).

5.2 Restaurant & Menu Module

Provides full CRUD REST APIs for restaurants and their menu items. The listing page fetches restaurants from the API and renders them as cards; selecting a restaurant loads its menu via a filtered query (?restaurantId=).

5.3 Cart & Order Module

Cart state is managed entirely client-side using React Context, allowing quantity adjustments before checkout. On placing an order, a document is created in the Order collection tied to the authenticated user, and an order history view lists all past orders with status.

5.4 Real-Time Tracking Module

Socket.io rooms are created per order (order_<id>). When an order's status changes — whether from a payment confirmation or an admin update — the server emits an event to that room, and the client's tracking page updates its progress indicator live.

5.5 Payment Module

Integrates Razorpay's order-and-verify flow: the backend creates a Razorpay order, the frontend opens Razorpay Checkout, and on completion the backend verifies the cryptographic signature (HMAC-SHA256) before marking the order as paid — preventing client-side tampering with payment status.

6. Sprint-wise Development Timeline

The project was developed iteratively over eight one-day sprints, each committed to GitHub upon completion:

Sprint	Date	Deliverable
1	Sep 1	Project setup, homepage & restaurant listing UI
2	Sep 2	Express server, MongoDB connection, REST APIs (users/restaurants/menu)
3	Sep 3	JWT authentication (signup, login, protected routes)
4	Sep 4	Frontend-backend integration (Axios, AuthContext)
5	Sep 5	Cart, order placement, order history
6	Sep 6	Real-time order tracking with Socket.io
7	Sep 7	Razorpay payment gateway integration, automated testing
8	Sep 8	Deployment configuration (Vercel + Render), final report

7. API Reference (Summary)

Method	Endpoint	Description
POST	/api/auth/signup	Register a new user, returns JWT
POST	/api/auth/login	Authenticate user, returns JWT
GET	/api/auth/me	Get logged-in user profile
GET/POST/PUT/DELETE	/api/users	User management (CRUD)
GET/POST/PUT/DELETE	/api/restaurants	Restaurant management (CRUD)
GET/POST/PUT/DELETE	/api/menu	Menu item management (CRUD)
POST	/api/orders	Place a new order (auth required)
GET	/api/orders/my	Get logged-in user's order history
GET	/api/orders/:id	Get single order detail
PUT	/api/orders/:id/status	Update order status (admin only)
POST	/api/payments/create-order	Create Razorpay payment order
POST	/api/payments/verify	Verify payment signature

8. Testing

Automated tests were written using Jest and Supertest, covering route validation, authentication enforcement, and JWT middleware behavior. All 14 tests pass.

Category	Test Case	Result
Health & routing	GET /api/health returns ok	Pass
Health & routing	GET unknown route returns 404	Pass
Auth validation	Signup without required fields returns 400	Pass
Auth validation	Login without required fields returns 400	Pass
Route protection	GET /api/auth/me with no token returns 401	Pass
Route protection	POST /api/orders with no token returns 401	Pass
Route protection	GET /api/orders/my with no token returns 401	Pass
Input validation	POST /api/restaurants missing fields returns 400	Pass
Input validation	POST /api/menu missing fields returns 400	Pass
JWT middleware	Rejects missing/invalid tokens	Pass
JWT middleware	Accepts valid token, attaches req.user	Pass
Role authorization	Blocks disallowed roles (403)	Pass
Role authorization	Allows permitted roles through	Pass

Manual testing was also performed across the full user journey: signup → browse restaurants → view menu → add to cart → place order → pay via Razorpay test mode → track order status changes in real time.

9. Deployment

The application is configured for cloud deployment:

- Frontend: deployed on Vercel, built with Vite (vercel.json handles SPA routing).
- Backend: deployed on Render as a Node web service (render.yaml defines the build/start commands and required environment variables).
- Database: hosted on MongoDB Atlas (free tier cluster).

Environment variables (MONGO_URI, JWT_SECRET, RAZORPAY_KEY_ID/SECRET, VITE_API_URL, VITE_SOCKET_URL) are kept out of source control and configured directly on each hosting platform.

10. Challenges Faced

- Keeping the Socket.io connection synchronized with REST-driven state changes (e.g., a payment verification triggering a live UI update) required careful event design using per-order rooms.
- Ensuring payment security meant verifying Razorpay's signature server-side rather than trusting the client's success callback.
- Structuring the Express app (app.js) separately from the server bootstrap (server.js) was necessary to make the API testable with Supertest without needing a live database or open port.

11. Conclusion

This project successfully demonstrates a complete full stack food delivery application, covering RESTful API design, secure authentication, real-time communication, payment processing, automated testing, and cloud deployment. It provided practical, hands-on experience with the MERN stack and with structuring a non-trivial application into clean, maintainable modules.

11.1 Future Enhancements

- Restaurant/admin dashboard for managing menus and order statuses

- Delivery partner tracking with live map/location updates
- Ratings and reviews for restaurants and menu items
- Push notifications for order status changes